

Can Nemotron Understand Geometry?

A Midcurve Experiment with Nemotron 3 4B Nano

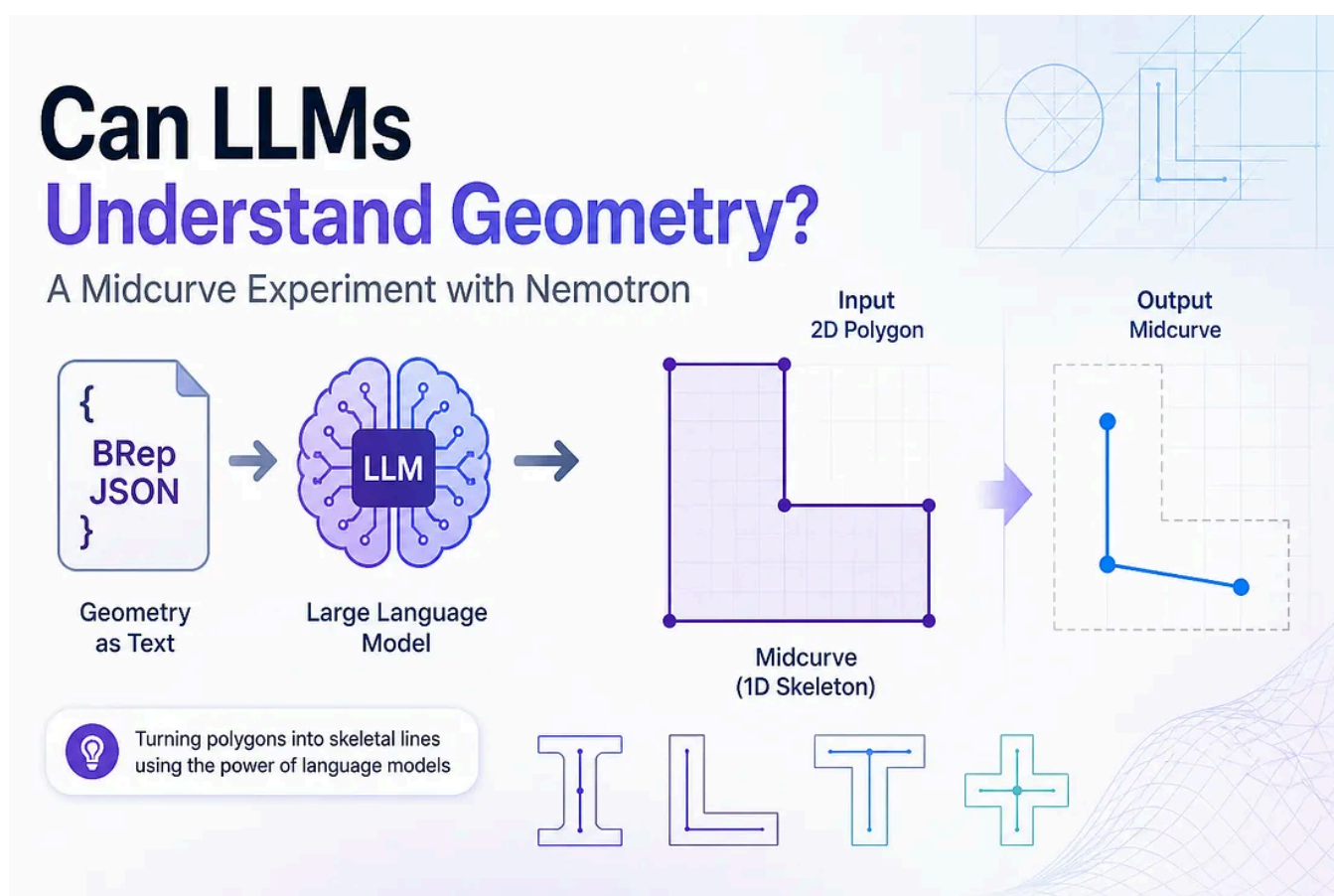
10 min read · Just now



Yogesh Haribhau Kulkarni (PhD)



... More

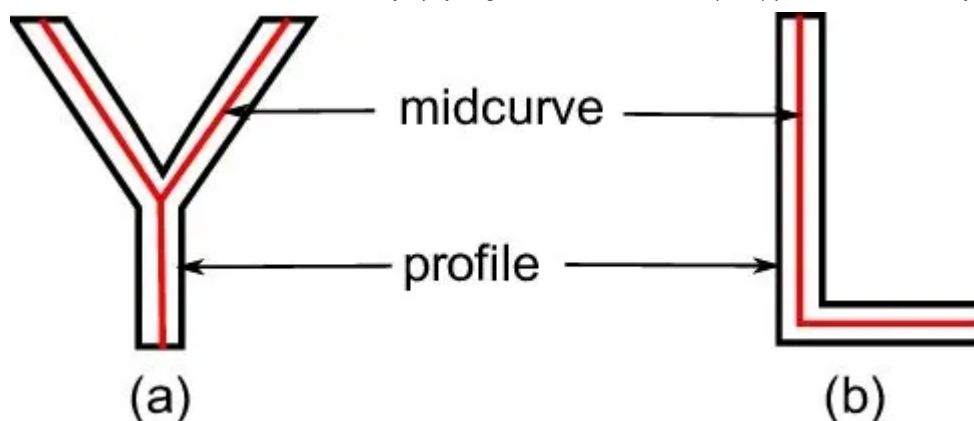


(Generated using ChatGPT by supplying detailed prompt)

The Midcurve Problem

In mechanical CAD, thin-walled structural components, I-beams, L-brackets, T-sections, etc. are routinely simplified for finite element analysis by collapsing their full 3D volume into a lower-dimensional equivalents.

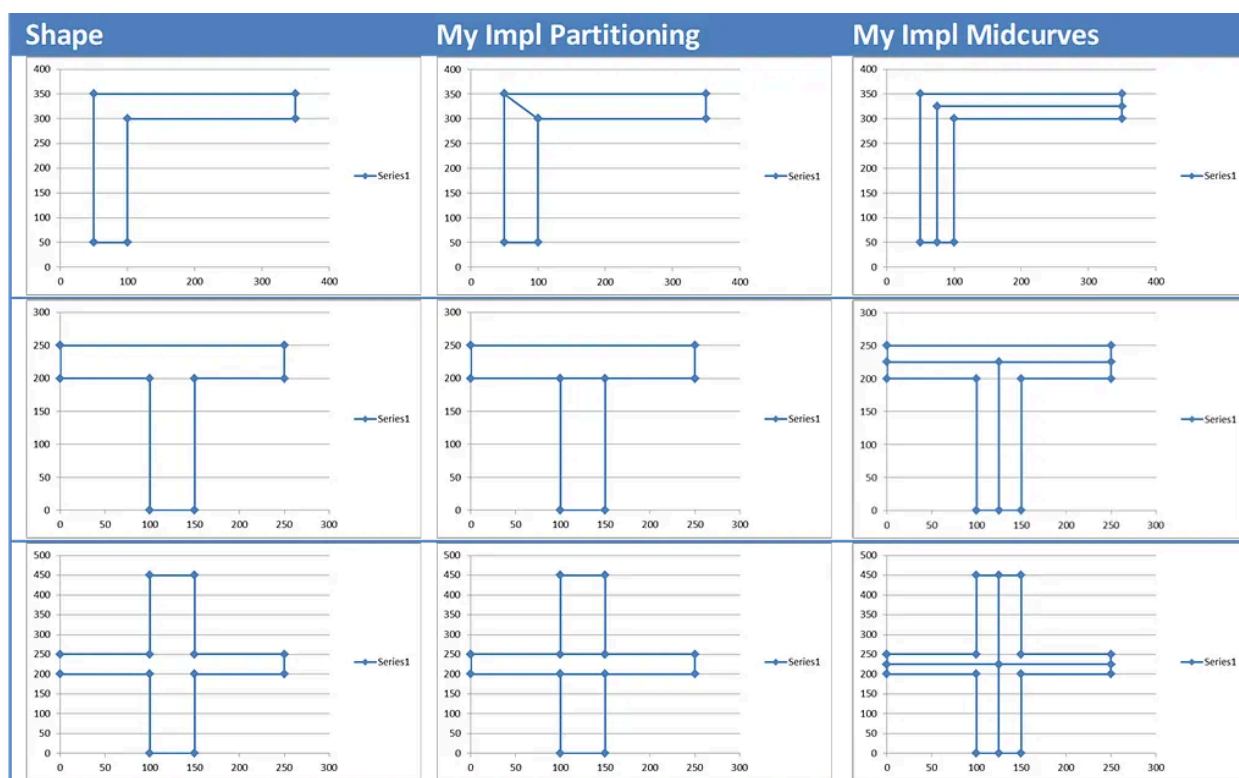
The 2D equivalent of this operation is computing the “midcurve”, a 1D polyline (or branched graph) that runs through the geometric center of a 2D closed polygon profile, equidistant from its bounding edges.



(Source: [MidcurveNN Github repository](#))

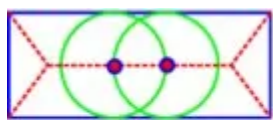
Formally, given a closed 2D polygon (a set of connected line segments), the task is to produce its midcurve, which may itself be a branched graph, not a simple open polyline. Think of it as the **skeleton of a shape**.

- An **I-shape** collapses to a single line
- An **L-shape** becomes two connected segments
- A **T-shape** forms a junction
- A **Plus** produces four branches

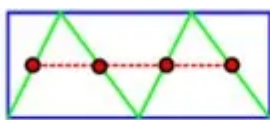


(Source: [MidcurveNN Github repository](#))

Despite decades of research, Medial Axis Transform, Chordal Axis Transform, thinning, pairing, the problem remains unsolved in the general case. Shape complexity and the variety of junction types make rule-based approaches brittle.



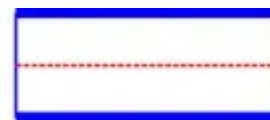
MAT



CAT



Thinning



Pairs

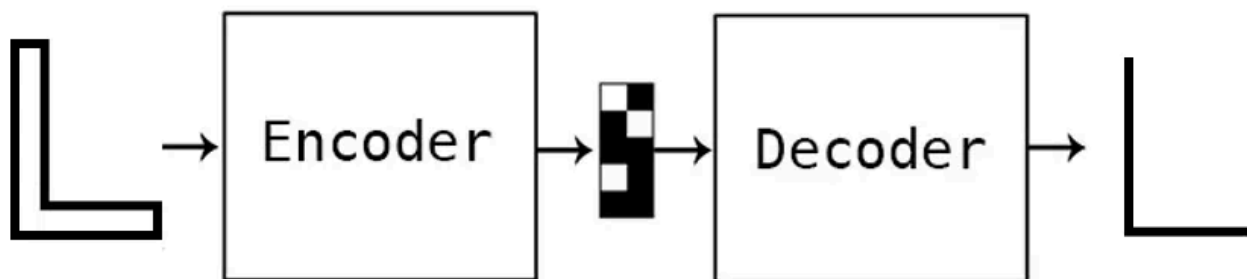
(Source: [MidcurveNN Github repository](#))

Three Approaches in MidcurveNN

The [MidcurveNN](#) research project evaluates neural network strategies across three paradigms:

Image-Based

Polygons are rasterized to 256×256 bitmaps and fed to encoder-decoder architectures: a simple dense encoder-decoder (baseline), CNN encoder-decoder, UNet with 8 encoding levels and CoordConv, and Pix2Pix GANs. The UNet variant performs best among image-based models.



(Source: [MidcurveNN Github repository](#))

Limitation: Rasterization loses exact geometry. Post-processing is needed to extract clean polylines from predicted bitmaps, and junction topology is blurred at pixel boundaries.

Text-Based (LLM)

If geometry can be 'serialized as text', large language models pre-trained on code and structured data may learn the pattern. This is the focus of the current article.

Geometry-Based (Graph Transformer)

A non-auto-regressive Graph Transformer trained from scratch treats polygon corners as graph nodes (features: x, y coordinates) and predicts the midcurve adjacency matrix along with output coordinates. This handles branching natively and is the strongest approach to date.

Why Text-Based?

Image-based methods fundamentally cannot represent exact geometry, and graph-based models require custom architectures. LLMs, however, are pre-trained on vast code corpora including JSON, SVG, and structured numeric formats. The insight: if shapes are expressed as compact JSON structures, fine-tuning an LLM is equivalent to teaching it a geometric transformation rule, similar to how code models learn API patterns.

Early experiments (2023) with few-shot prompting of GPT-4, Claude, Bard, and Llama showed that frontier models can sometimes compute midcurves of simple shapes from in-context examples, while smaller models fail consistently. This motivates **few short prompt examples** showing how the task can be posed as structured text transformation.

BRep JSON: Encoding Geometry as Text

The key design choice is the “Boundary Representation (BRep) JSON” format. Each shape is a dict with three parallel arrays:

```
You are a geometric transformation program that transforms input 2D polygonal
Input 2D polygonal profile is defined by Boundary Representation (Brep) format

{
  'ShapeName': 'I',
  'Profile': [(5.0, 5.0), (10.0, 5.0), (10.0, 20.0), (5.0, 20.0)],
  'Midcurve': [(7.5, 5.0), (7.5, 20.0)],
  'Profile_brep': {
    'Points': [(5.0, 5.0), (10.0, 5.0), (10.0, 20.0), (5.0, 20.0)], # list of
    'Lines': [[0, 1], [1, 2], [2, 3], [3, 0]], # list of point ids (ie index
    'Segments': [[0, 1, 2, 3]] # list of line ids (ie index in Li
  },
  'Midcurve_brep': {
    'Points': [(7.5, 5.0), (7.5, 20.0)],
    'Lines': [[0, 1]],
    'Segments': [[0]]
  },
}
```

where,

- ShapeName (text): name of the shape. Just for visualization/reports.
- Profile (text): List of Points coordinates (x,y) representing outer profile
- Midcurve (text): List of Points coordinates (x,y) representing inner midcurve
- Profile_brep (text): Dictionary in Brep format representing outer profile
- Midcurve_brep (text): Dictionary in Brep format representing inner midcurve

Each Segment is a continuous list of lines. In case of, say `Midcurve-T`, as

Below are some example transformations, specified as examples shapes.

Shape 1:

```
{
  'ShapeName': 'I_rotated_72',
  :
}
```

Shape 2:

```
{
  'ShapeName': 'Plus_translated_-30',
  :
}
```

Shape for prediction:

```
{
  'ShapeName': 'L_rotated_172',
  'Profile': [[-5.65, -4.26], [-10.6, -3.56], [-14.08, -28.32], [-38.83, -24.8],
  'Midcurve': <to predict>
  'Profile_brep': {
    'Points': [[-5.65, -4.26], [-10.6, -3.56], [-14.08, -28.32], [-38.83, -24.8],
    'Lines': [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 0]],
    'Segments': [[0, 1, 2, 3, 4, 5]]},
  'Midcurve_brep': <to predict>
}
```

- **Points:** carry the geometry (x, y coordinates).
- **Lines:** carry the topology (index pairs).
- **Segments:** group lines into topological faces or branches.

Key Insight: Geometric transformations modify only coordinates (Points).
Topology (Lines, Segments) remains invariant.

This makes the task look like a **structured transformation problem**, something LLMs are good at.

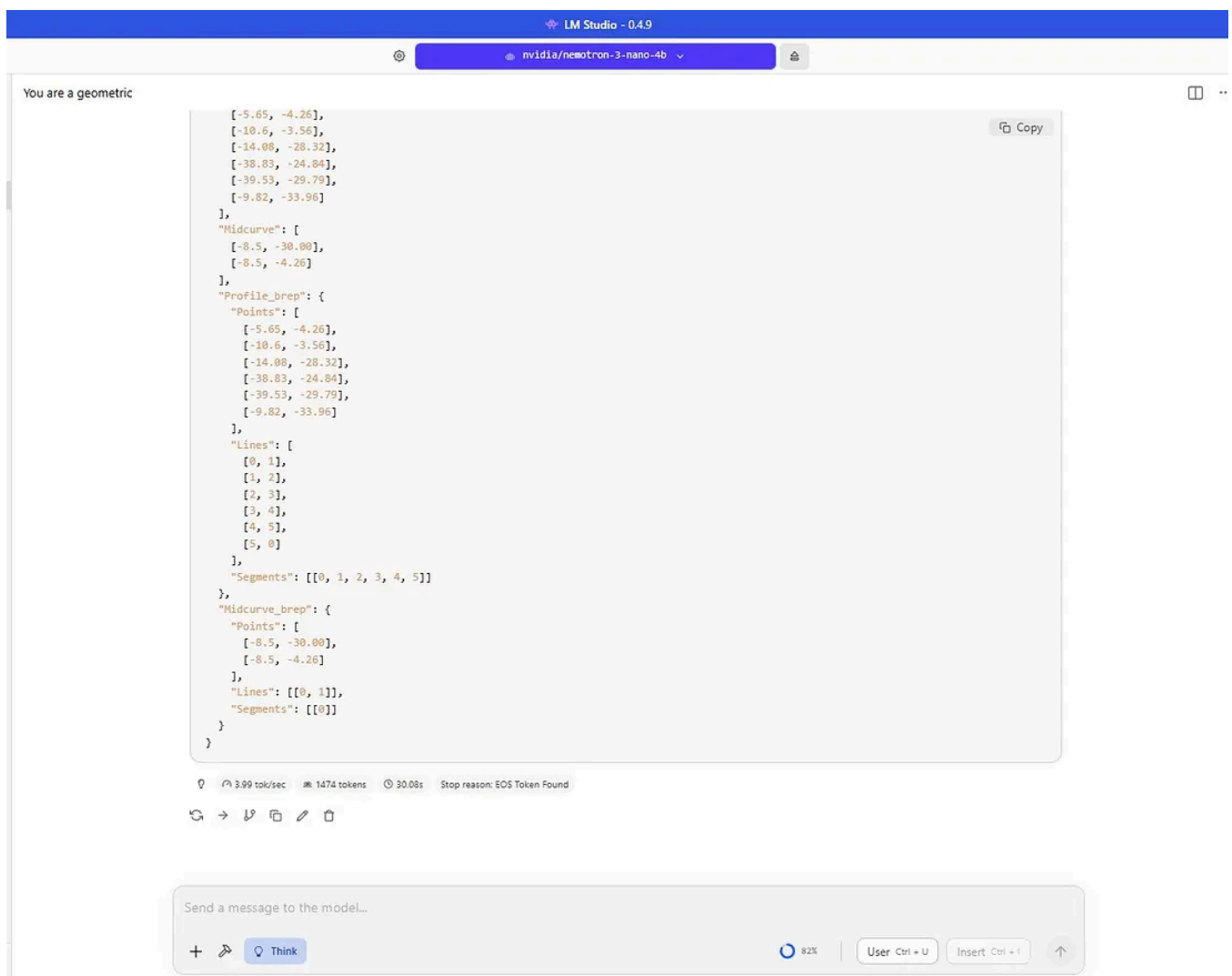
Experiment: Few-Shot Prompting

We tested this using LM Studio with an instruction-tuned Nemotron model. Instead of training, we used **few-shot prompting**:

- Provide 2 example transformations
- Ask the model to predict a third

The task: *Convert a 2D polygon (Profile) into its midcurve representation.*

LM Studio was used to run an instruction-tuned <https://lmstudio.ai/models/nvidia/nemotron-3-nano-4b> model locally. The model was loaded through the LM Studio interface, and inference was performed by directly supplying a structured prompt describing the geometric transformation task. The BRep JSON input was pasted into the prompt window, along with a few example transformations to guide the model. The model then generated the predicted midcurve in the same JSON format.



Screenshot by the Author

The prompt used for this experiment is shown below, followed by a comparison of actual vs predicted outputs.

You are a geometric transformation program that transforms input 2D polygonal Input 2D polygonal profile is defined by Boundary Representation (Brep) format

```
{
  'ShapeName': 'I',
  'Profile': [(5.0, 5.0), (10.0, 5.0), (10.0, 20.0), (5.0, 20.0)],
  'Midcurve': [(7.5, 5.0), (7.5, 20.0)],
  'Profile_brep': {
    'Points': [(5.0, 5.0), (10.0, 5.0), (10.0, 20.0), (5.0, 20.0)], # list of
    'Lines': [[0, 1], [1, 2], [2, 3], [3, 0]], # list of point ids (ie index
    'Segments': [[0, 1, 2, 3]] # list of line ids (ie index in Li
  },
  'Midcurve_brep': {
    'Points': [(7.5, 5.0), (7.5, 20.0)],
    'Lines': [[0, 1]],
    'Segments': [[0]]
  },
}
```

where,

- ShapeName (text): name of the shape. Just for visualization/reports.
- Profile (text): List of Points coordinates (x,y) representing outer profile
- Midcurve (text): List of Points coordinates (x,y) representing inner midcurve
- Profile_brep (text): Dictionary in Brep format representing outer profile segments
- Midcurve_brep (text): Dictionary in Brep format representing inner midcurve segments

Each Segment is a continuous list of lines. In case of, say `Midcurve-T`, as

Below are some example transformations, specified as examples shapes.

Shape 1:

```
{
  'ShapeName': 'I_rotated_72',
  :
}
```

Shape 2:

```
{
  'ShapeName': 'Plus_translated_-30',
  :
}
```

Shape for prediction:

```
{
  'ShapeName': 'L_rotated_172',
  'Profile': [[-5.65, -4.26], [-10.6, -3.56], [-14.08, -28.32], [-38.83, -24.8
  'Midcurve': <to predict>
  'Profile_brep': {
```



```

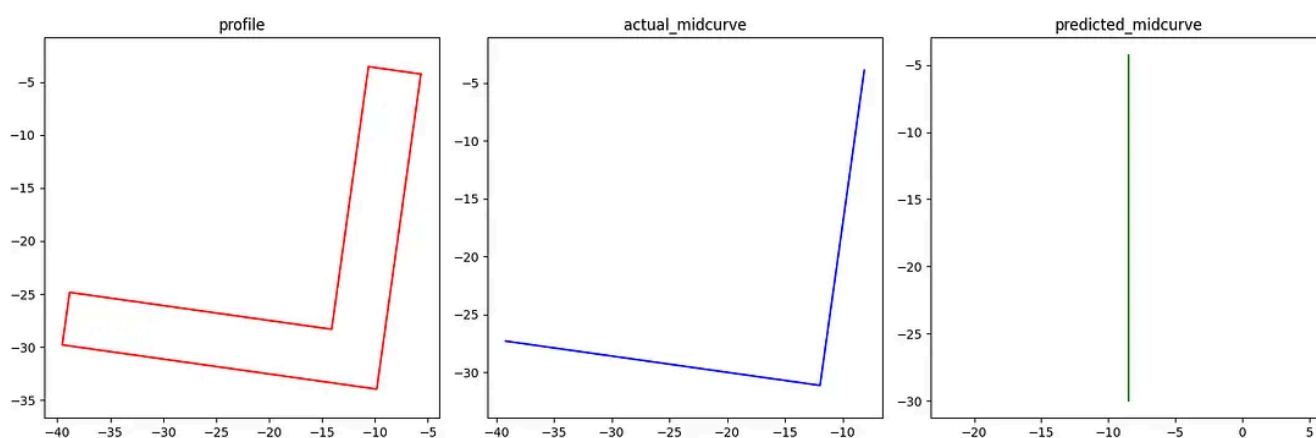
'Points': [[-5.65, -4.26], [-10.6, -3.56], [-14.08, -28.32], [-38.83,
'Lines': [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 0]],
'Segments': [[0, 1, 2, 3, 4, 5]]},
'Midcurve_brep': <to predict>
}

```

Results

Using a prompt-based approach in LM Studio, the model was able to generate structurally valid BRep JSON for the test shape, preserving schema consistency (Points, Lines, Segments) without any post-processing. This indicates that instruction-tuned LLMs can reliably follow structured output formats when geometry is serialized clearly.

However, a comparison between the **actual** and **predicted** midcurves reveals significant geometric inaccuracies. The expected output captures the true medial structure of the rotated 'L' shape with a bent polyline reflecting both arms of the geometry. In contrast, the model prediction collapses the solution into a single straight segment, missing the corner transition and failing to represent the underlying topology.



(Results generated by the Author)

This highlights a key limitation: while the model understands the *format* and partially the *intent*, it does not yet capture precise geometric reasoning such as symmetry, thickness distribution, or junction formation. The prediction is syntactically correct but geometrically incomplete.

Why Does It Fail?

This exposes a deeper limitation of LLMs. They are excellent at:

- pattern matching
- format replication
- statistical inference

But midcurve computation requires:

- symmetry detection
- thickness reasoning
- spatial invariants
- topology preservation

These are **geometric priors**, not just patterns in text. *The model understands* what the output should look like *but not* what it should mean geometrically.

Why This Matters

Midcurves are not just academic:

- Used in **finite element analysis (FEA)**
- Critical for **mesh generation**
- Enable **lightweight geometric modeling**

Failure here signals a broader limitation: *LLMs are not yet reliable for **physical or spatial reasoning tasks**.*

Conclusion

This experiment reinforces an important and somewhat counterintuitive insight: while large language models can *operate* on structured geometric data, they do not yet *reason* about geometry in a physically meaningful way. By framing the midcurve problem as a BRep JSON transformation, we effectively translated a spatial problem into a textual one and the model responded well at the surface level. It adhered to schema constraints, maintained structural consistency, and produced outputs that appeared plausible at a glance.

However, a deeper inspection reveals a fundamental gap. The model does not internalize geometric principles such as symmetry, medial axis formation, or

thickness distribution. Instead, it relies on learned statistical patterns from examples, which breaks down when the task demands true spatial understanding. The incorrect midcurve is not just a numerical deviation, it reflects a failure to preserve topology and capture the intrinsic structure of the shape.

This highlights a broader limitation of current LLMs: they lack the inductive biases required for **physical-geometric reasoning**. Tasks like midcurve extraction are governed by invariants and spatial relationships that are not easily inferred from text alone. While prompt-based approaches demonstrate feasibility and accessibility, they also expose the ceiling of what can be achieved without incorporating geometry-aware mechanisms.

For organizations like NVIDIA, this represents both a challenge and an opportunity. Advancing in this domain will likely require hybrid approaches that combine the flexibility of LLMs with the rigor of geometric or graph-based models. Embedding spatial priors, leveraging differentiable geometry, or integrating symbolic reasoning could be key steps toward bridging this gap. Until then, LLMs remain powerful assistants for formatting and approximating geometry but not authoritative engines for computing it.

Final Thought

What makes this exploration particularly compelling is how close and yet how far — we are. With nothing more than a carefully designed prompt and a few examples, the model produces outputs that *look* correct, follow all structural rules, and give the illusion of understanding. This is a testament to the remarkable generalization ability of modern LLMs.

And yet, the illusion breaks the moment we examine the geometry itself. The model does not “see” the shape. It does not recognize the bend in an L, the junction in a T, or the symmetry in a Plus. It processes tokens, not space. The result is a system that can convincingly mimic geometric reasoning without actually performing it.

This gap is not a failure, it is a signal. A signal that the next frontier in AI is not just scaling models or refining prompts, but fundamentally rethinking how machines represent and reason about the physical world. Bridging language and geometry, syntax and structure, pattern and principle, this is where meaningful progress lies.

Until then, prompt-based methods offer a fascinating glimpse into what is possible today, while reminding us of the deeper challenges that remain unsolved.

Midcurve

Fine Tuning

LLM

Lora

Geometry



Following

Published in Pune AI Community

13 followers · Last published just now

The Pune AI Community (PAIC) is a collaborative platform dedicated to making AI knowledge accessible from classes to masses. We bring together AI engineers, researchers, students, startups, and professionals to learn, discuss, build, and grow together.



Edit profile

Written by Yogesh Haribhau Kulkarni (PhD)

1.9K followers · 2.1K following

PhD in Geometric Modeling | Google Developer Expert (Machine Learning) | Top Writer 3x (Medium) |
More at <https://www.linkedin.com/in/yogeshkulkarni/>